

— final year major project, notes to self —

Smart Campus Agent System

Agentic AI-driven Campus Operations Platform

Dept. of CSE · NMAM Institute of Technology · 2026
Manvith Y Shetty · M vaibhav · Suján S · Mohith L A

IN ONE LINE

A campus back-office that runs itself: AI agents build the timetable with a real constraint solver, and when a teacher takes leave they re-plan the affected classes on their own — a human only clicks Approve.

NOT a chatbot

it acts on events,
not only on prompts

NOT one big prompt

supervisor + specialists
+ typed tools + solver

NOT auto-pilot

every real change passes
a human approval gate

What's in these notes

- p2 · The problem & the big idea
- p3 · Bird's-eye architecture
- p4 · Tech stack (and why each)
- p5 · Data model — the 16 tables
- p6 · The agent brain (LangGraph)
- p7 · Flow A — timetable generation
- p8 · Inside the CP-SAT solver

- p9 · Flow B — leave → period exchange
- p10 · The flagship flow, step by step
- p11 · How a partner is chosen (scoring)
- p12 · Human-in-the-loop: pause & resume
- p13 · Proactive triggers & resilience
- p14 · API + screens map
- p15 · Status, roadmap, viva ammo

Two flows carry the demo:

- A. generate a clash-free timetable (solver)
- B. a leave repairs itself (agents + human gate)

Remember the shape of the whole thing:

DETECT → PLAN → APPROVE
→ ACT → NOTIFY

1 · The problem we picked

Every college runs on three painful manual jobs:

Timetabling

Weeks of clash-hunting on a whiteboard / Excel. One change breaks three other things.

Teacher absence

HOD phones around at 8 a.m. Students get a free period or a stand-in teaching the wrong subject.

Bookings & info

Halls double-booked; circular info lives in 40 WhatsApp groups.

Why an "agent", not an app?

A normal app waits to be told. An agent observes an event, decides on its own what work that creates, does it, and asks a human only for the final yes/no.

CHATBOT

user asks →
text comes back

≠

OUR SYSTEM

event happens → agent plans →
human approves → DB changes + people notified

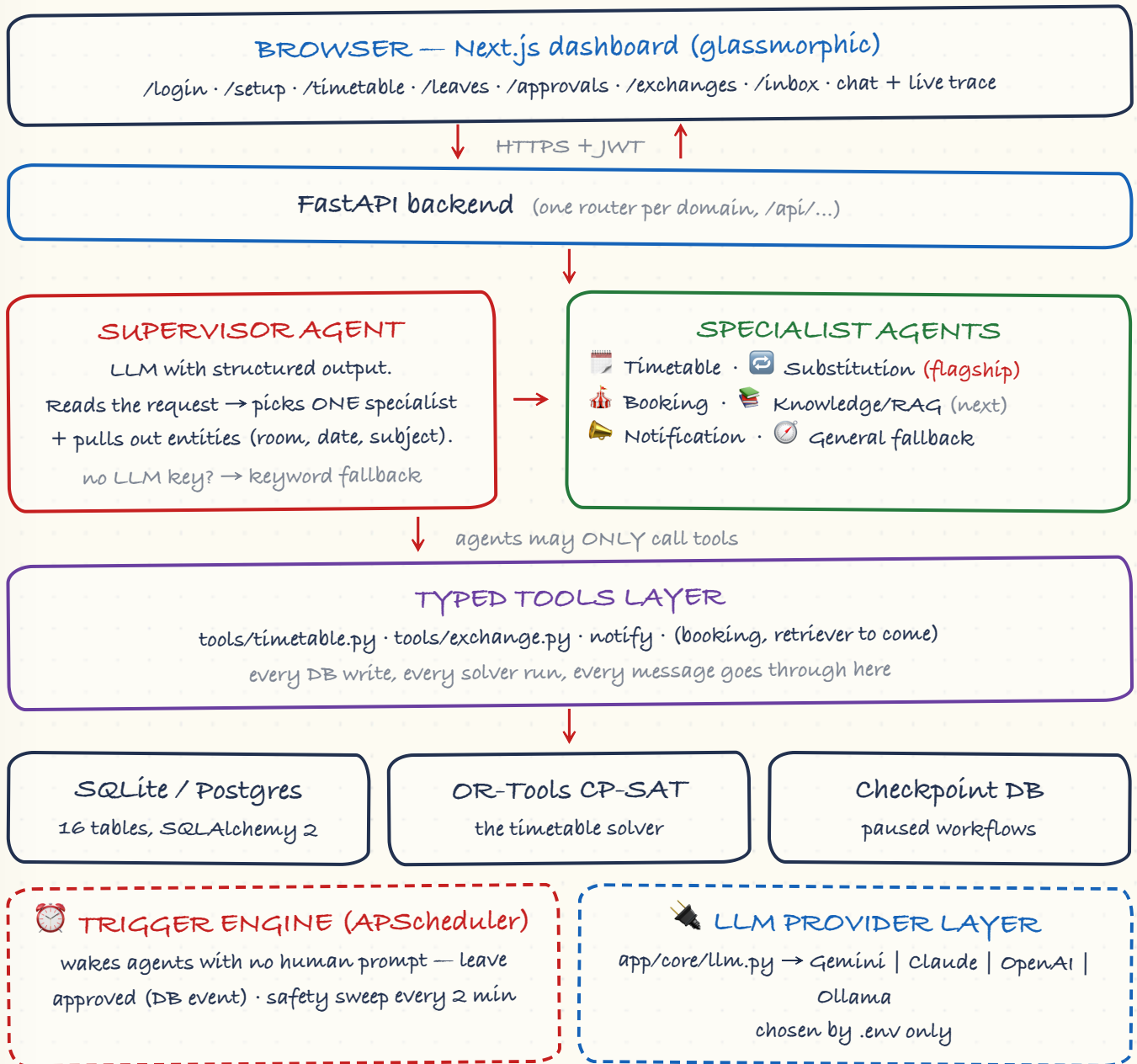
2 · The big idea (3 design rules)

- 1 Right tool for the right job. The LLM *never* solves the timetable — timetabling is NP-hard and an LLM can only guess. A CP-SAT constraint solver does it and can prove it is clash-free. The LLM is used only where language is involved: understanding the request, explaining the result.
- 2 Agents never touch the database directly. Every capability is a typed tool (a Python function with a schema). The model may only *call* tools. So every action is logged, permission-checked and auditable — and a hallucination cannot corrupt data.
- 3 Humans hold the switch. The agent proposes; a person approves. Implemented properly — the workflow literally pauses mid-execution and resumes days later from the same point.

Fourth rule (architecture flex): no vendor lock-in. Gemini / Claude / GPT / a local Ollama model — switching is one line in .env, zero code change. The intelligence is in the design, not in somebody's API key.

3 · Bird's-eye architecture

read it top-to-bottom: who asks → who decides → who does → where it's stored



TWO DOORS into the same brain: ① a human types in chat → POST /api/agent/chat ② something happens (leave approved, cron tick) → the system injects a task itself. Same graph, same agents, no user needed. ← this is the "agentic" bit.

4 · Tech stack — and why each piece

chosen for: free for students · runs on a laptop · industry standard · no lock-in

Layer	Choice	Why this one
Frontend	Next.js 14 + TS + Tailwind	Dashboard + live agent-trace sidebar; industry standard.
Backend	FastAPI + Uvicorn (Py 3.11)	Async, auto Swagger docs, WebSockets ready.
Agent orchestration	LangGraph	The reason we can pause a workflow — supervisor pattern, interrupt(), durable checkpoint.
Timetable solving	Google OR-Tools CP-SAT	Constraint solving is a solver's job. Proves clash-freeness; explains infeasibility.
LLM	provider-agnostic layer	Default Gemini Flash (free tier). Swap by .env.
Database	SQLite → Postgres	Zero setup for 4 teammates; swap = connection string.
ORM	SQLAlchemy 2.x	Typed models; create_all for now (Alembic later).
Background jobs	APScheduler	In-process cron — no Redis/Celery at this scale.
Auth	JWT + pbkdf2	Roles: admin / faculty / student, checked per route.
Vector store (Ph.3)	ChromaDB (embedded)	For the RAG knowledge assistant — zero infra.

★ The model-agnostic LLM layer — app/core/llm.py

One factory. Every agent asks it for a model. Nothing else knows who the vendor is.

```
def get_llm(purpose="default"):
    return init_chat_model(
        model = settings.LLM_MODEL,          # "gemini-flash-latest"
        model_provider = settings.LLM_PROVIDER, # google_genai|anthropic|openai|ollama
        temperature = 0.1)
```

Switching provider = editing .env

```
LLM_PROVIDER=anthropic
LLM_MODEL=claude-sonnet-5
ANTHROPIC_API_KEY=...
```

And if the key is missing / quota dies?

The supervisor falls back to deterministic keyword routing. Ollama can run the whole thing offline. The demo cannot be killed by the venue Wi-Fi.

5 · Data model — 16 tables (SQLAlchemy)

grouped by what they're for; arrows = foreign keys that matter

PEOPLE & AUTH

users (name, email, role, hash)
teachers (dept, max_hours_per_day)
teacher_subjects ← who can teach what

ACADEMIC STRUCTURE

subjects (code, periods_per_week, needs_lab)
sections · rooms (classroom | lab | hall...)
timeslots (MON-FRI × P1-P7 = 35)

↓ the solver writes into ↓

timetable_entries — the master timetable

section · subject · teacher · room · timeslot · status · version
every solver run writes a NEW version → old ones kept → history + diffs

LEAVE & SUBSTITUTION

leaves (teacher, from, to, reason, status)
period_exchanges ← the live model
substitutions (legacy v1, kept as history)
timetable_configs (rules that made version N)

WORKFLOW & OUTPUT

approvals (kind, ref_id, status,
langgraph_thread_id)
notifications (user, title, body, read)
events · bookings · documents (Phase 3)

Two design decisions worth defending

① timetable_entries.version — nothing is ever overwritten. Regenerating makes $v(n+1)$; the UI reads the latest. So a bad generation is never destructive.

② approvals.langgraph_thread_id — this single column is the bridge between a human clicking a button in a browser and a paused Python workflow. The click looks up the thread and resumes it. Explain this in the viva; it's the cleverest column in the schema.

Why period_exchanges is a separate table and not a change to the timetable

Exchanges are a dated overlay. The master timetable is never mutated — we verified 312 rows → 312 rows across a full approve cycle. A leave on one Tuesday must not rewrite the permanent schedule.

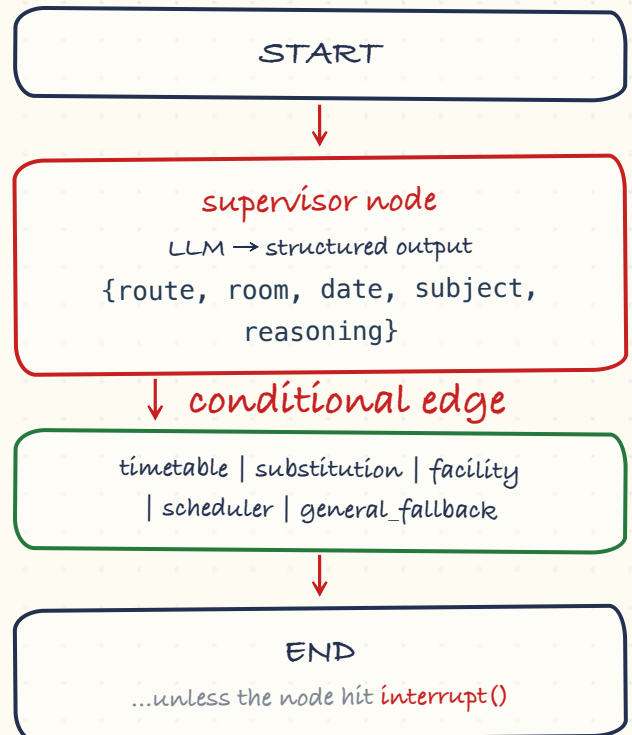
Seeded demo data (auto, idempotent, on startup): CSE dept · 2 sections (CSE-7A/7B) · 6 teachers · 8 subjects · 8 rooms · 35 timeslots · 3 demo logins.

6 · The agent brain — LangGraph

The whole "AI" part is a state machine whose nodes are agents. One shared state object flows through it and each node adds to it.

AgentState (the shared clipboard)

```
messages      # the conversation
current_action # route chosen
steps         # trace list (UI!)
params        # results
final_response # text answer
source        # "user" | "system"
task_spec     # entities, leave_id...
```



Why "structured output" matters

The supervisor doesn't return prose that we then regex. It returns a Pydantic object — the route is a `Literal[...]`, so it is impossible for the LLM to invent a specialist that doesn't exist. Validation happens before any code runs.

3 routing paths, in priority order

1. **System trigger short-circuit** — if `source=="system"` and a `leave_id` is present → go straight to `substitution`, no LLM call at all (the trigger already knows the intent). This alone took the flagship flow from ~7 s to ~72 ms.
2. **LLM routing** — normal chat.
3. **Keyword fallback** — provider down / no key.

The checkpointer (`SqliteSaver → checkpoints.db`)

After every node, the full state is written to disk against a `thread_id`. Consequences: conversations survive a restart, and a workflow can be frozen mid-node and thawed later — which is exactly what human-in-the-loop needs (see p12).

Adding a new agent is 3 lines — write `specialists/x.py`, `add_node("x", fn) + add_edge("x", END)`, add the route name to the supervisor's `Literal`. No keyword lists to maintain. ← say this when they ask about extensibility

7 · Flow A — Timetable generation (F1)

- ① Admin enters master data once, at /setup — subjects, teachers (+ which subjects each can teach), sections, rooms. Forms or CSV import with per-row error reporting. 17 endpoints; reads = anyone, writes = admin.
- ② Trigger: the *Generate* button (POST /api/timetable/generate, admin-only) or chat: "generate a fresh timetable" → supervisor routes to the Timetable Agent.
- ③ The tool assembles solver input from the DB — load_timetable_input() turns SQLAlchemy rows into plain dataclasses (sections, subjects, teachers, rooms, slots). Solver stays decoupled from the ORM.
- ④ *Precheck first* — cheap arithmetic sanity tests before the solver runs, because they give the precise reason: "Section CSE-7A needs 26 periods/week but only 24 slots", "No teacher can teach CS701 — map one in Data Setup."
- ⑤ *CP-SAT solves it* — 8 hard constraints + optional admin rules + a fairness objective. ~8 s for the demo campus, provably clash-free (see p8).
- ⑥ Store as a new version — lessons written to timetable_entries with version = n+1; the rules used are saved in timetable_configs so you can always answer "which rules produced v7?".
- ⑦ UI renders the MON-FRI × P1-P7 grid at /timetable — per-subject colours, lab blocks marked, teacher + room in each cell, Download PDF (client-side jsPDF, any role).

If it's impossible — the part that impresses

Most published timetable systems use genetic algorithms: when they fail they just... fail. Ours returns which constraint groups conflict, using CP-SAT assumption literals — each constraint family is guarded by a boolean, and on infeasibility the solver names the smallest guilty set:

HTTP 422

"Conflicting constraint groups: teacher daily hour limits, lab consecutive-block scheduling."

"Typical fixes: add teachers/rooms, raise max_hours_per_day, or reduce periods_per_week."

Admin-configurable rules (Phase 2.2 / 2.3)

Per class, not just globally. section_rules = global defaults + per-section overrides. One section can have a WED half-day while the others keep the full day (implemented as per-section allowed slot sets).

Defaults matter. "No same subject back-to-back" is ON by default — hectic scheduling should be prevented, not opted into. The chat path loads the admin's saved config too, so regenerating from chat keeps the rules.

8 · Inside the CP-SAT model

app/solver/timetable_model.py — the deterministic heart of the project

Decision variables (all boolean)

$x[\text{section}, \text{subject}, \text{slot}]$ = is this lesson placed here?

$y[\text{section}, \text{subject}, \text{teacher}]$ = who teaches this pairing?

$w = x \text{ AND } y$ = teacher actually busy at that slot (needed for clash + load rules)

$c[\text{section}, \text{room}]$ = the section's home classroom · $l_r[\dots]$ = which lab room

Hard constraints — must ALL hold

#	Rule
H1	every (section, subject) gets <i>exactly</i> its periods_per_week
H2	a section has at most one lesson per timeslot (students can't be in 2 places)
H3	a teacher teaches at most one lesson per timeslot (the clash rule)
H4	exactly one <i>qualified</i> teacher per (section, subject)
H5	labs = one block of consecutive same-day periods, in a lab room; lab rooms clash-free
H6	each section gets a dedicated home classroom with enough capacity
H7	teacher daily load $\leq$ max_hours_per_day
H8	at most 2 periods of the same subject per day (spread)
H9	no same-subject back-to-back — optional, ON by default, per class
H10	cap on a teacher's consecutive teaching run — optional
+	half-days per class — e.g. "WED ends after P4", by slot filtering

Soft objective — fairness

minimize(busiest_teacher_load - idlest_teacher_load)

A hard constraint says "legal"; the objective says "fair". Measured load gap on the demo data: 2 periods.

verified

48 lessons, zero clashes by an independent
SQL check, ~8 s

vs. the literature

GA-based papers: slower, no optimality proof,
no explanation on failure

Adjacency is time-aware, not index-aware. "Back-to-back" means `prev.end == next.start` — a lunch break *breaks* adjacency. The exchange planner (p11) reuses the exact same definition, so the two subsystems agree.

9 · Flow B — Leave & Substitution (THE FLAGSHIP)

First, the domain insight that makes this project real

v1 did what textbooks do: find a free teacher to cover the class. But that means a stand-in teaching an unfamiliar subject — useless for students. Real colleges exchange periods. So we rebuilt it (Phase 2.1).

The exchange, in one picture

Teacher **A (Anita)** is on leave — TUE 21 Jul. She has CS704 with CSE-7B at TUE P5.

TUE P5 (leave day)

was: A teaches CS704

now: **B (Suresh)** teaches

his own CS701

Thursday's lesson pulled forward



THU P2 (recovery day)

was: B teaches CS701

now: **A** teaches

her own CS704

Tuesday's lesson recovered

Net effect: every subject keeps its weekly hours · the right teacher always teaches the right subject · only two dates differ · the master timetable is untouched.

Why no student clash is even possible here

Both sides of the swap are lessons of the same section — so that section's occupancy at every slot is unchanged. Clash-freeness is guaranteed by construction, not by checking. (Nice thing to say out loud.)

The hard cases are surfaced, not hidden

- No partner free → row says **"NO EXCHANGE AVAILABLE — needs manual arrangement"**.
 - Lab lesson → flagged for manual handling (splitting a consecutive lab block breaks it).
- The HOD sees these before approving. Human stays in control.

Q: "why not just let AI assign a substitute?" → Because the college doesn't work that way, and a wrong-subject stand-in helps nobody. Building what the user actually does beats building what the paper says. This change is the strongest "we studied the real domain" evidence in the project.

10 · The flagship flow, step by step

nobody types a single prompt after step 2 — that's the whole point

- ① Faculty applies for leave at /leaves → POST /api/leaves → row in leaves, status pending.
- ② Admin/HOD clicks Approve → POST /api/leaves/{id}/decide. Status → **approved**.
← this is the last human action for a while.
- ③ The API itself invokes the agent graph — a synthetic message with source="system", task_spec={leave_id}, a fresh thread_id. Nobody asked it to. This is the proactive trigger.
- ④ Supervisor short-circuits — system source + leave_id → route substitution with no LLM round-trip.
- ⑤ Substitution Agent loads the leave, checks it really is approved, and calls build_plan() — see p11 for the algorithm. Rows land in period_exchanges as proposed, grouped by plan_id = "exchange-leave-<id>".
- ⑥ An Approval row is created (kind=substitution_plan) carrying the langgraph_thread_id.
- ⑦ interrupt() — the graph freezes mid-node. State is checkpointed to disk. The HTTP call returns immediately with the proposed plan; the workflow is now just... waiting. Days, if needed.
- ⑧ HOD opens /approvals and sees a card: leave date/slot · class · missed subject · partner teaches (own subject) · recovery date/slot · why. Rows with a ⚠ warning are tinted amber.
- ⑨ Approve → POST /api/approvals/{id}/decide → looks up the thread id → graph.invoke(Command(resume={"action":"approve"}), thread_id) → **execution continues from the exact line after interrupt()**.
- ⑩ apply_plan() — exchanges → confirmed; notifications written to both the partner ("on 21 Jul you teach CS701 to CSE-7B in place of Anita; your THU P2 goes to her") and the returning teacher (her recovery schedule).
- ⑪ Everyone sees it — /inbox, the /exchanges board, and the effective day view (/api/timetable/effective/CSE-7B?date=...) which overlays confirmed exchanges onto that day's grid, marked ⇌ amber.
- ⑫ Reject instead? → reject_plan() — all rows rejected, no notifications, overlay stays clean.

MEASURED: leave-approval → plan on screen ≈ **72 ms** (was ~7 s before the Phase 2.3 fixes)
two culprits killed: an unnecessary LLM call on system triggers + an N+1 query storm in the planner

11 · How a partner is chosen — `build_plan()`

`app/tools/exchange.py` · pure Python + SQL, no LLM anywhere in here

Step 1 — what's affected?

Latest-version timetable entries of teacher A whose weekday falls inside `[from_date, to_date]`
→ each becomes one lesson to repair, on a concrete date D.



Step 2 — who could be the partner B? (all must hold)

- ✓ B teaches the same section S (that's what makes it a clean swap)
- ✓ B is free at A's slot on date D — no lesson, not on leave, not already used by another exchange
- ✓ a recovery date R exists: the nearest date after the leave ends whose weekday = B's slot day...
- ✓ ...and A is free at that slot on R, and the (partner slot, R) pair isn't already taken
- ✓ labs excluded (consecutive block — handled manually)



Step 3 — score every candidate, take the best

```
score = -100.0 × (adjacency warnings)  ← avoid hectic days
        - 2.0 × (R - D).days           ← nearest recovery (syllabus pacing)
        - 0.5 × (swaps already given to B in this plan)  ← spread the load
        - 0.1 × (B's weekly lesson count)               ← fairness
```

The **-100 penalty is deliberately soft, not a hard reject**: a clean swap always beats a hectic one, but if every option is hectic the plan is still produced — carrying a Δ warning the HOD can see. Never silently give up on a class.

What counts as "hectic"? (the Phase 2.2 adjacency guard)

same subject
back-to-back
for students

> 2 periods of a
subject in one day

3+ consecutive teaching
periods for either teacher

Checked against an effective picture of that date — base timetable + other confirmed exchanges + placements made earlier in this same plan.

Two engineering details worth marks

PlanContext — all DB reads done once (~5 queries) into dicts/sets; every check after that is in-memory. Replacing the naïve per-candidate queries was validated by a differential test: 0 mismatches across all sections/teachers/days.

Idempotency — `build_plan` returns early if rows for that `plan_id` exist. Essential, because a resumed LangGraph node **re-executes from the top** (verified: ran twice → 2 rows, not 4).

12 · Human-in-the-loop — pause & resume

the mechanism most projects fake with a status flag; ours actually suspends execution

INSIDE THE AGENT NODE

```
rows      = build_plan(db, leave)
summary   = plan_summary(db, id)
approval  = Approval(..., thread_id)
```

```
decision = interrupt({
    "type": "substitution_plan",
    "approval_id": approval.id,
    "plan": summary })
```

```
# — resumes HERE, later —
if decision["action"]=="approve":
    apply_plan(db, leave_id)
else:
    reject_plan(db, leave_id)
```

WHAT HAPPENS AT 🙋

1. LangGraph raises a special interrupt
2. Checkpointer writes the full state to checkpoints.db under this thread_id
3. invoke() returns with __interrupt__ holding the proposed plan
4. FastAPI answers the HTTP request normally
5. The Python process keeps nothing in memory. You may now restart the server, reboot the laptop, come back on Monday.

WHAT HAPPENS ON APPROVE

Command(resume={"action":"approve"}) + the same thread_id → LangGraph reloads the checkpoint, re-runs the node, and interrupt() returns the decision instead of pausing. Execution continues at the next line.

⚠️ The consequence you must remember (and mention)

Because the node re-runs from the top on resume, everything before interrupt() must be safe to run twice — hence the idempotent build_plan and the "does a pending Approval already exist?" check. This is the classic gotcha of durable workflows; showing you handled it deliberately is worth marks.

Where the human sits in the loop



Safe autonomy is a live research theme — an agent that proposes and a human who disposes. Low-risk actions (sending a notification) don't need a gate; anything that changes the schedule does. That policy is a design decision, and you should say so.

13 · Proactive triggers & resilience

What can wake an agent up

Trigger	Kind	Wakes
Leave approved	DB / API event	Substitution Agent
Safety sweep, every 2 min	APScheduler cron	Substitution Agent
Booking pending > 24 h	cron sweep	Notification Agent (Ph.3)
Weekly attendance scan	cron	Attendance Sentinel (stretch)

The safety sweep — a small piece of code with a big story

Every 2 minutes: "is there an approved leave with no plan rows?" If yes, trigger the agent for it. So even if the server died in the exact millisecond between "leave approved" and "plan built", the system heals itself. Self-healing, not just fault-tolerant.

Every failure mode, and what happens

If this breaks...	...the system does this
No LLM key / bad key	Supervisor uses deterministic keyword routing; everything still works
LLM provider down / quota	Exception caught → same keyword fallback, noted in the trace
Internet down at the venue	Switch .env to Ollama — fully offline
Server restarts mid-approval	Checkpoint + langgraph_thread_id → resume still works
Trigger lost entirely	APScheduler sweep re-triggers within 2 min
Node re-runs on resume	Idempotent plan building → no duplicate rows
Timetable is impossible	422 + plain-language reasons + suggested fixes
No partner free for a swap	Row marked "NO EXCHANGE AVAILABLE"; the rest of the plan still applies

Say this in the viva: "every one of these degrades gracefully — the demo can't hard-fail. If Gemini errors live on stage, that becomes the resilience demo."

Security rails

- JWT bearer + role checks per route (require_role("admin")) — a student's chat cannot mutate a timetable
- Tools enforce permissions, not the LLM
- Analytics SQL tool (planned) will be SELECT-only, validated, row-limited
- Every tool call is written into steps → shown live in the UI trace → full audit trail

14 · API surface & screens

full list always live at localhost:8000/docs (Swagger) — this is the map

agent · auth

GET /api/health
POST /api/agent/chat
POST /api/auth/login
GET /api/auth/me

setup (17 endpoints)

CRUD for subjects · teachers · teacher↔subject
map · sections · rooms
+ CSV import / template download
reads: any user · writes: admin

timetable

POST /timetable/generate (admin)
GET /timetable/status
GET /timetable/section/{name}
GET /timetable/teacher/{id}
GET /timetable/effective/{sec}?date=
GET /timetable/exchanges?from=&to=

leaves · approvals · inbox

POST /leaves GET /leaves
POST /leaves/{id}/decide (admin)
GET /approvals (admin)
POST /approvals/{id}/decide (admin)
GET /notifications
GET /notifications/unread_count
POST /notifications/{id}/read

The screens (Next.js app router)

Page	What it's for
/login	JWT auth, role-aware redirect
/ (dashboard)	overview + agent chat with the live workflow-trace sidebar
/setup	master data: tabbed CRUD + CSV
/timetable	MON-FRI × P1-P7 grid, Constraints panel, Generate, Download PDF
/leaves	faculty apply · admin approve/reject
/approvals	the HOD's plan cards — the human gate
/exchanges	exchange board + effective (dated) day view
/inbox	notifications, 15 s polling, unread badges

Demo logins (seeded on startup)

admin@campus.edu / admin123 · anita.rao@campus.edu / faculty123 ·
student@campus.edu / student123

Run it

```
cd services/backend && python main.py      # :8000, auto-seeds  
cd apps/web      && npm run dev            # :3000
```

15 · Where we are & what's left

DONE ✓

- ✓ Ph 0 — foundation: LangGraph, DB + seed, JWT, LLM layer, supervisor
- ✓ Ph 1 — F1 timetable: CP-SAT, setup UI, grid UI, infeasibility explanations
- ✓ Ph 2 — F2 leave & substitution + HTL + inbox
- ✓ Ph 2.1 — period-exchange model + dated overlay + /exchanges
- ✓ Ph 2.2 — anti-hectic guard + configurable generation
- ✓ Ph 2.3 — per-class rules · back-to-back off by default · F2 ms plans · PDF export

NEXT □

- Ph 3 — F3 Event & venue booking (conflicts vs bookings and the timetable, approval chain, 24 h nag)
- Ph 3 — F4 Knowledge RAG: PDF → Chroma → answers with citations
- WebSocket push instead of polling
- Ph 4 — pick stretch: F9 Analytics (text→safe SQL), F6 Attendance Sentinel, F7 Complaint triage, F8 Exam scheduler, F10 Energy watchdog
- Harden: CORS, rate limits, error states
- Final report + diagrams + rehearsed demo

Viva ammunition — memorise these five

Q "Why not let the LLM make the timetable?"

NP-hard; an LLM cannot guarantee clash-freeness. CP-SAT proves it and explains failure. The LLM parses intent and explains outcomes — each tool does what it is provably good at.

Q "Isn't this just one prompt to ChatGPT?"

No — supervisor-router multi-agent graph, typed tools, a constraint solver, a versioned DB, and human-in-the-loop interrupts. The LLM is one swappable line of .env; remove it and the flagship flow still runs.

Q "What makes it agentic?"

Nobody prompts it. A leave approval event wakes the agent; it decides what work that creates, plans it, pauses for a human, then acts and notifies. Detect → plan → approve → act → notify.

Q "What if the server restarts mid-approval?"

The workflow is checkpointed in SQLite and the approval row stores the thread id — resume works after a restart. Plus a 2-minute sweep re-triggers any approved leave that lost its plan.

Q "What's genuinely novel?"

(i) proactive event-driven multi-agent operation rather than chat; (ii) a solver + LLM split where the LLM explains infeasibility; (iii) the period-exchange model taken from how the college actually runs, with a fairness- and fatigue-aware planner; (iv) true durable human-in-the-loop. No published campus system combines these.

If you remember one sentence:

status & viva

"The system observes an event, plans the work, waits for one human click, then acts